Nombre: Cesar Maximiliano Peña Mendoza Diego Alberto Sierra Mendoza Arturo Sebastian Pérez Zapata	**Matrícula:** 2144135 2143733 2142942
Nombre del curso: COMPONENTES DE LOGICA DE NEGOCIOS	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo Netacad: 5	**Actividad:** ACT05-U5. LABORATORIO GUIADO
Fecha: 05/11/2026	
Bibliografía (Formato APA) Beck, K. (2003). Test-driven development: By example. Addison-Wesley. GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions pytest-dev. (s. f.). pytest documentation. https://docs.pytest.org SQLAlchemy. (s. f.). SQLAlchemy documentation. https://docs.sqlalchemy.org Astral. (s. f.). Ruff documentation. https://docs.astral.sh/ruff/	

Objetivo:

El objetivo de esta actividad es aplicar pruebas automatizadas con pytest, desarrollo guiado por pruebas (TDD), persistencia con commit y rollback, mocks de servicios externos, pruebas de autorización, medición de cobertura e integración continua con GitHub Actions sobre un proyecto de pedidos con reglas de negocio, y trasladar lo aprendido a las reglas reales de NovaRent, la empresa del PIA.

Procedimiento:

El proyecto implementa pedidos con reglas de negocio, persistencia con SQLAlchemy (SQLite en memoria), una pasarela de pago simulada con mocks, una política de cancelación y un pipeline de integración continua. A continuación se presentan el resumen de resultados, la arquitectura, los hallazgos, las evidencias E1 a E9, las matrices de casos y el enlace del repositorio.

1. Resumen de resultados

Verificación	Resultado	Evidencia
Pruebas del proyecto	47 PASSED y 1 XFAIL esperada (proyecto base: 13)	E1, E7
Cobertura de líneas sobre `src/`	100 % (55 sentencias; umbral 80 %)	E7
`ruff check src tests`	Sin errores	E1, E7
Matriz de casos (`datos/matriz_casos_prueba.csv`)	20 casos: 7 éxito, 9 error, 4 límite; todos PASSED	Sección 7
Aplicación al PIA (`pia_novarent/`)	20 casos (6 éxito, 6 error, 8 límite), 31 pruebas PASSED, cobertura 100 %	E9
Pipeline en GitHub Actions	Ejecuciones verde, roja y verde	E8

Verificación	Resultado	Evidencia
Detección de defectos	8 pruebas fallan al romper el descuento VIP; 3 casos fallan al romper reglas de NovaRent	E1, E9

2. Estructura del repositorio

Ruta	Contenido
`src/`	Dominio, servicio de cobro, repositorio y política de cancelación
`tests/`	Unitarias (`test_order`, `test_security`), integración (`test_repository`, `test_integration`), mocks (`test_service`) y casos desde CSV (`test_casos_csv`)
`datos/`	CSV didácticos y `matriz_casos_prueba.csv` (20 casos con resultado real)
`evidencias/`	Salidas reales de cada paso (E1–E8), capturas y reporte HTML de cobertura
`pia_novarent/`	Aplicación al PIA (E9): código, pruebas, matriz y `pyproject.toml` propios
`.github/workflows/ci.yml`	ruff → pytest + cobertura ≥ 80 % → compileall → artefacto `htmlcov`; segundo job para `pia_novarent`
`scripts/ci_local.sh`	Simulación local de los mismos pasos del pipeline
`EVIDENCIAS_U5.md`	Bitácora E1–E9 con explicaciones

3. Arquitectura y diagramas

3.1 Arquitectura por capas

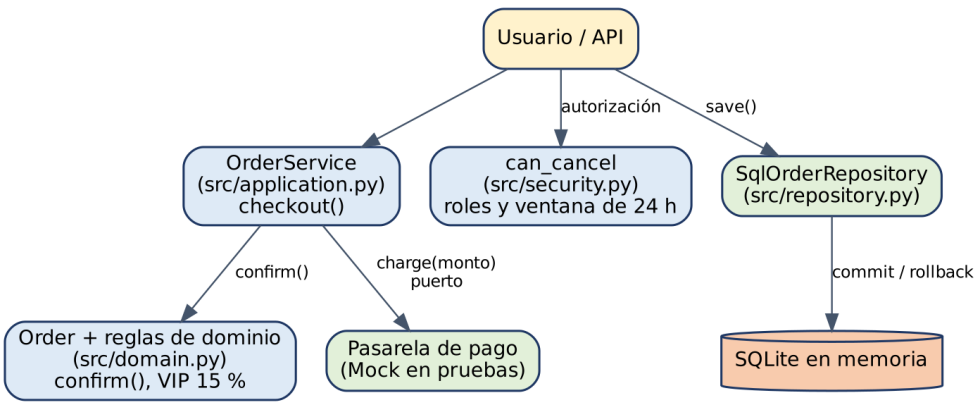


Figura 1. Arquitectura del proyecto: servicio, dominio, puertos y persistencia

Componente	Archivo	Responsabilidad	Cómo se prueba
Dominio `Order`	`src/domain.py`	Confirmar pedidos, descuento VIP de 15 %, validaciones (total positivo, sin deuda, no confirmar dos veces)	Unitarias, sin base de datos
Servicio `OrderService`	`src/application.py`	`checkout`: confirma y cobra mediante la pasarela	Mock de la pasarela

Componente	Archivo	Responsabilidad	Cómo se prueba
Repositorio	`src/repository.py`	`SqlOrderRepository` y `OrderRow` con SQLAlchemy	SQLite en memoria
Política de cancelación	`src/security.py`	`can_cancel`: reglas por rol, estado y antigüedad	Unitarias con fecha fija

El dominio no importa SQLAlchemy ni bibliotecas de pago. Las pruebas unitarias no usan base de datos; las de integración usan SQLite en memoria; la pasarela siempre es un mock, por lo que nunca hay cobros reales.

3.2 Estados del pedido

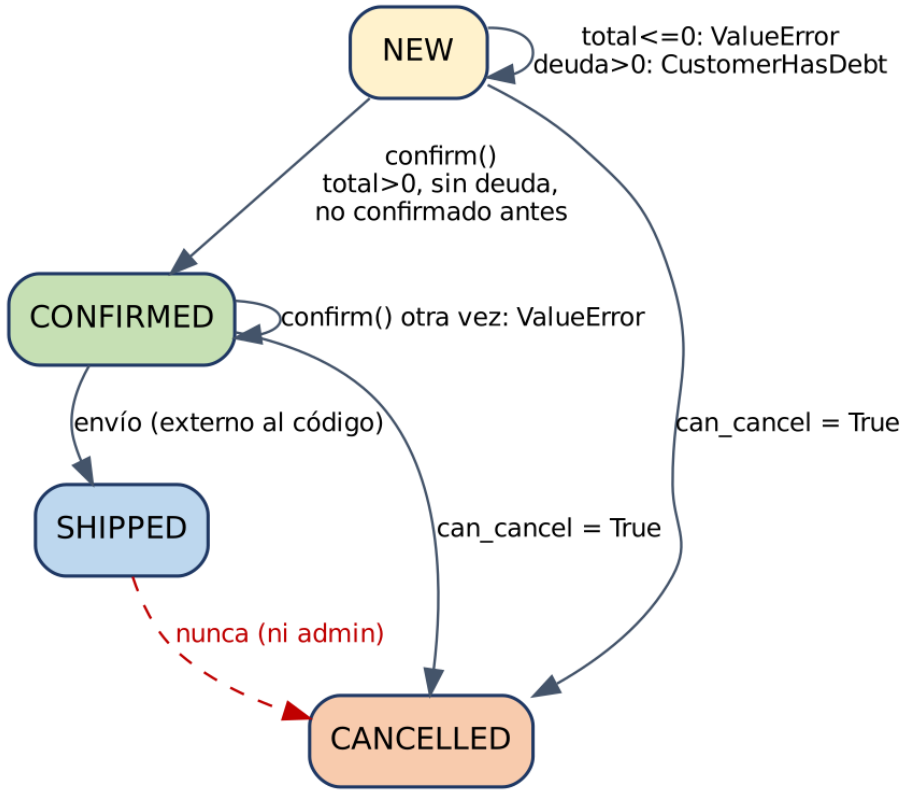


Figura 2. Estados del pedido y transiciones permitidas o rechazadas

Transición	Condición	Resultado
NEW → CONFIRMED	`total > 0`, sin deuda y no confirmado antes	Éxito; cobro con descuento VIP si aplica
CONFIRMED → CONFIRMED	Segundo `confirm()`	Error: `ValueError("already confirmed")`
NEW → NEW	Total ≤ 0 o deuda > 0	Error: `ValueError` o `CustomerHasDebt`; el estado no cambia
NEW / CONFIRMED → cancelación	`can_cancel` devuelve True	Permitido (administrador o dueño dentro de 24 h)

Transición	Condición	Resultado
SHIPPED → cancelación	Cualquier rol	Error: denegado, incluso para administrador

3.3 Decisión de autorización

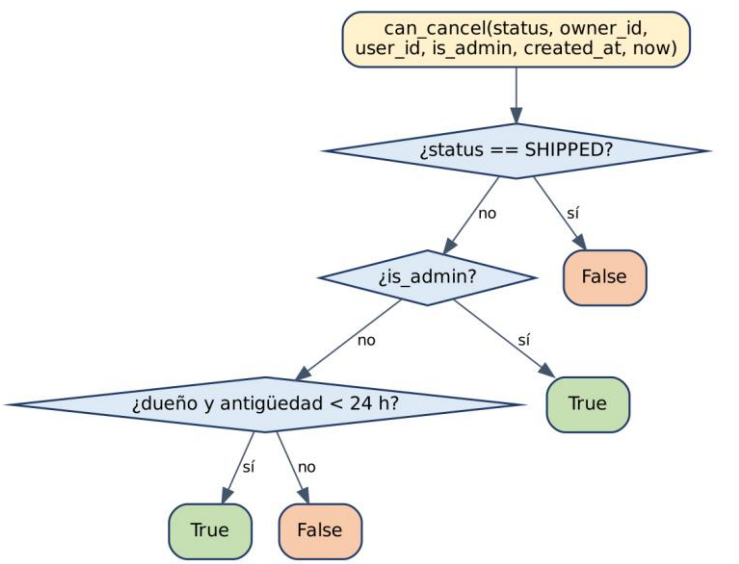


Figura 3. Flujo de decisión de can_cancel

El orden de las condiciones importa: `SHIPPED` se evalúa primero y tiene prioridad sobre el rol. Después se evalúa el administrador y, por último, la regla del dueño con la ventana estricta de menos de 24 horas.

4. Hallazgos del proyecto base

#	Hallazgo	Evidencia	Acción
1	El CI ejecuta `pytest` y no `python -m pytest`; así `src` no es importable	`ModuleNotFoundError: No module named 'src'` en las 4 suites	`pythonpath = ["."]` en `pyproject.toml`
2	`requirements.txt` permite `ruff>=0.6`; con ruff 0.17.3 el código base no pasa el lint	`E1b_ruff_base.txt`: 12 errores (I001, UP017)	`ruff check --fix`; se recomienda fijar la versión de ruff
3	`checkout` confirma el pedido antes de saber si el pago fue aceptado	`test_rejected_payment_should_not_leave_order_confirmed` (XFAIL)	Propuestas A, B y C en E5

#	Hallazgo	Evidencia	Acción
4	CP08 del CSV describe el comportamiento previo al TDD (confirmaba dos veces)	`ESPERADO_TRAS_TDD` en `tests/test_casos_csv.py`	Esperado actualizado a `ValueError`

Cambios y decisiones de diseño

Cambio	Motivo
`pythonpath = ["."]` en `pyproject.toml`	El CI ejecuta `pytest` (sin `python -m`) y fallaba con `ModuleNotFoundError: src`
Imports ordenados y `datetime.UTC`	`ruff` 0.17 marcaba 12 errores en el código base
Regla `already confirmed` en `Order.confirm()`	Ciclo TDD (E3)
`xfail(strict=True)` en `test_rejected_payment_should_not_leave_order_confirmed`	Documenta el riesgo de consistencia de `checkout` sin ocultarlo; si alguien lo corrige, la prueba obliga a quitar la marca
Regla de cancelación	`SHIPPED` nunca se cancela, ni por administrador (prioridad sobre el rol)

5. Tipos de prueba en este proyecto

Tipo	Qué prueba	Dependencias reales	Archivos
Unitaria	Reglas de `Order` y `can_cancel`	Ninguna	`test_order.py`, `test_security.py`, `test_casos_csv.py`
Integración	`OrderService` con mock de pasarela y `SqlOrderRepository` con SQLite en memoria	Base de datos de prueba	`test_repository.py`, `test_service.py`, `test_integration.py`
E2E	No existe en el proyecto base (no hay API ni interfaz)	—	Solo escenario propuesto

Escenario E2E propuesto (no implementado)	Paso
1	Un cliente VIP crea un pedido de 200 por la API
2	El sistema cobra 170 con la pasarela de pruebas
3	El pedido queda CONFIRMED en una base temporal
4	El dueño lo cancela dentro de las primeras 24 h

6. Evidencias de ejecución

E1 · Instalación y ejecución

Comando	`python -m pytest -v`
Resultado	47 passed, 1 xfailed (la salida se capturó en dos imágenes consecutivas)
Explicación	`pytest` descubre las funciones `test_*` y reporta cuáles cumplen sus aserciones. El entorno virtual (`venv`) aísla las dependencias del curso para que el resultado sea reproducible en otra máquina.

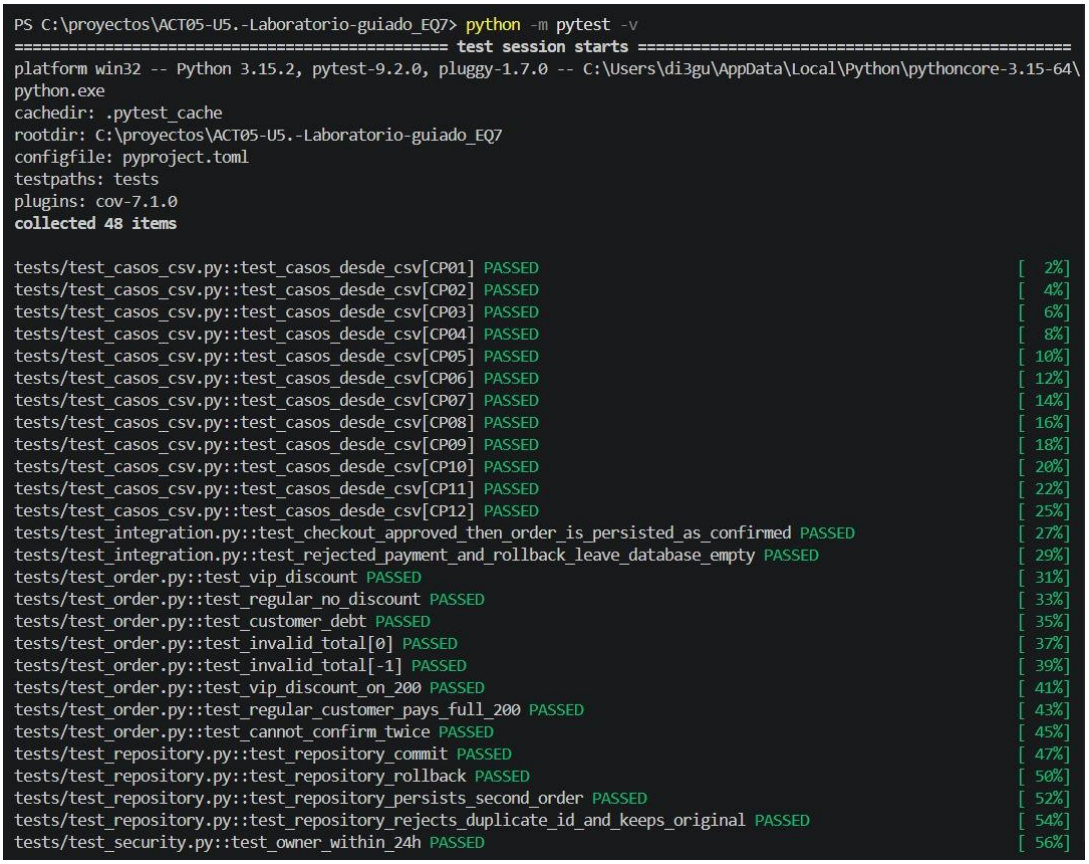


Figura 4. E1 – pytest, primera parte

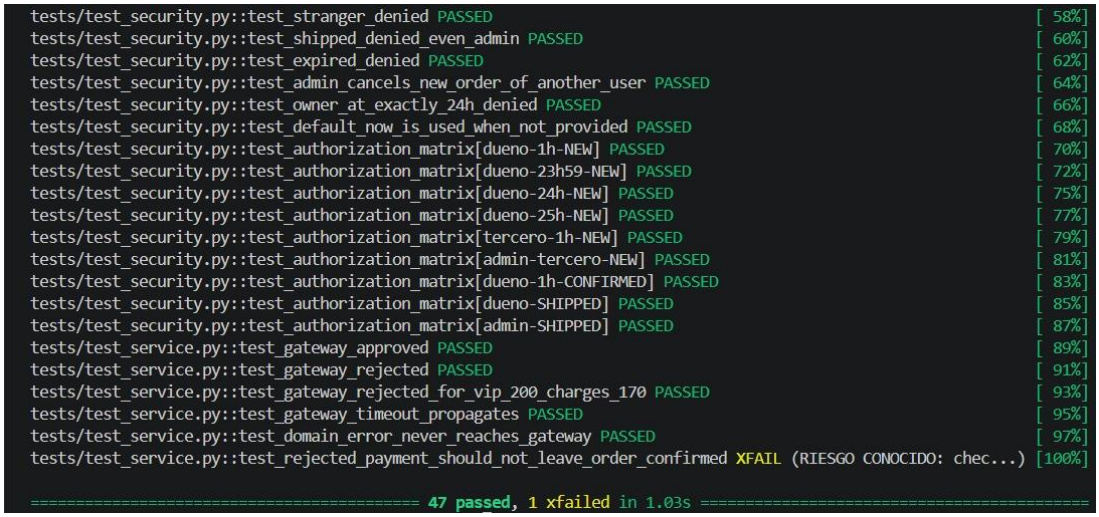


Figura 5. E1 – pytest, segunda parte y resumen: 47 passed, 1 xfailed

Verificación adicional	Qué ocurrió	Resultado	Archivo
Ruff en el proyecto base	Con ruff 0.17.3 el código original marcaba `I001` (imports sin ordenar) y `UP017` (alias `datetime.UTC`)	12 errores; corregidos con `ruff check --fix`	`E1b_ruff_base.txt`
CI corregido	Con `pythonpath = ["."]` el CI deja de fallar por importación	13 passed	`E1c_pytest_ci_corregido.txt`
La suite detecta defectos	Se inyectó un defecto a propósito: `VIP_DISCOUNT_RATE` de 0.15 a 0.10	8 failed, 39 passed, 1 xfailed (entre ellas `test_vip_discount` y `test_vip_discount_on_200`)	`E1d_defecto_detectado.txt`

E2 · Pruebas unitarias nuevas (Arrange–Act–Assert)

```
def test_vip_discount_on_200():
    # Arrange
    order = Order(200, vip=True)
    # Act
    charge = order.confirm()
    # Assert
    assert charge == 170
    assert order.discount == 30
    assert order.status == "CONFIRMED"

def test_regular_customer_pays_full_200():
    order = Order(200)
    assert order.confirm() == 200
    assert order.discount == 0
    assert order.status == "CONFIRMED"
```

Entrada	Regla	Resultado esperado	Resultado real
Total 200, VIP	Descuento VIP 15 %	Cobro 170, descuento 30, CONFIRMED	PASSED

Entrada	Regla	Resultado esperado	Resultado real
Total 200, regular	Sin descuento	Cobro 200, descuento 0, CONFIRMED	PASSED

Comando: `python -m pytest tests/test_order.py -v`

```
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7> python -m pytest tests/test_order.py -v
============================= test session starts =============================
platform win32 -- Python 3.15.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\
python.exe
cachedir: .pytest_cache
rootdir: C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7
configfile: pyproject.toml
plugins: cov-7.1.0
collected 8 items

tests/test_order.py::test_vip_discount PASSED                            [ 12%]
tests/test_order.py::test_regular_no_discount PASSED                     [ 25%]
tests/test_order.py::test_customer_debt PASSED                           [ 37%]
tests/test_order.py::test_invalid_total[0] PASSED                        [ 50%]
tests/test_order.py::test_invalid_total[-1] PASSED                       [ 62%]
tests/test_order.py::test_vip_discount_on_200 PASSED                     [ 75%]
tests/test_order.py::test_regular_customer_pays_full_200 PASSED          [ 87%]
tests/test_order.py::test_cannot_confirm_twice PASSED                    [100%]

============================== 8 passed in 0.10s ==============================
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7>
```

Figura 6. E2 – pruebas de dominio, 8 passed

E3 · TDD: Red → Green → Refactor

Regla de negocio: no permitir confirmar un pedido que ya ha sido confirmado previamente (`already confirmed`).

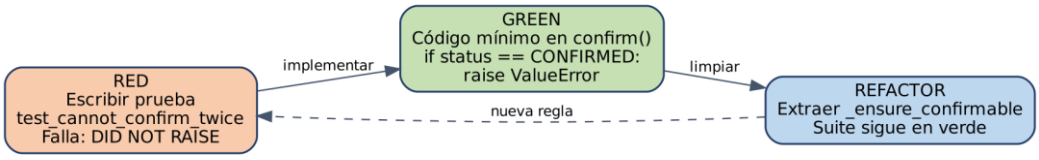


Figura 7. Ciclo TDD aplicado a la regla de confirmación doble

1. Fase RED (fallo controlado)

La prueba unitaria se diseña y ejecuta antes de implementar la validación en el dominio (`src/domain.py`). Código de la prueba (`tests/test_order.py`):

```
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()
    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()
```

Dato	Valor
Entorno de ejecución	pytest-9.2.0, pluggy-1.7.0, cov-7.1.0 en win32 (directorio raíz: `C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7`)
Archivo y línea evaluada	`tests\test_order.py:45`
Resultado en consola	`FAILED tests/test_order.py::test_cannot_confirm_twice`

Dato	Valor
Causa técnica del fallo	`Failed: DID NOT RAISE ValueError`
Métrica	1 failed in 0.28s

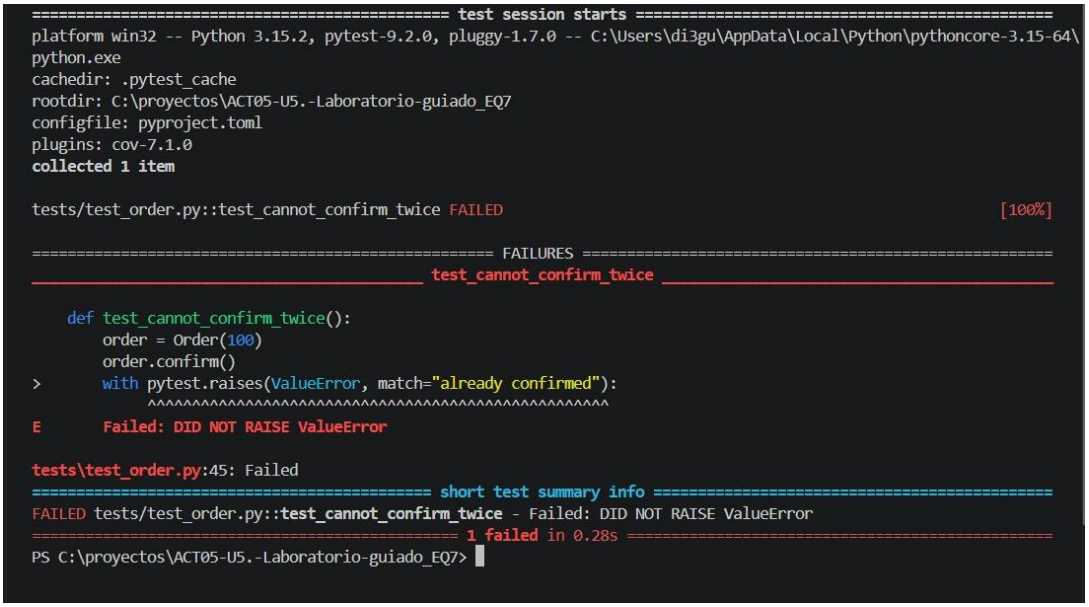


Figura 8. E3 – fase RED: DID NOT RAISE ValueError

2. Fase GREEN (implementación mínima funcional)

Se añade la lógica mínima indispensable al inicio del método `confirm()` en `src/domain.py` para satisfacer la prueba:

```
def confirm(self, customer_debt=0):
    if self.status == "CONFIRMED":
        raise ValueError("already confirmed")
    ...
```

Dato	Valor
Resultado en consola	`tests/test_order.py::test_cannot_confirm_twice PASSED [100%]`
Métrica	1 passed in 0.08s

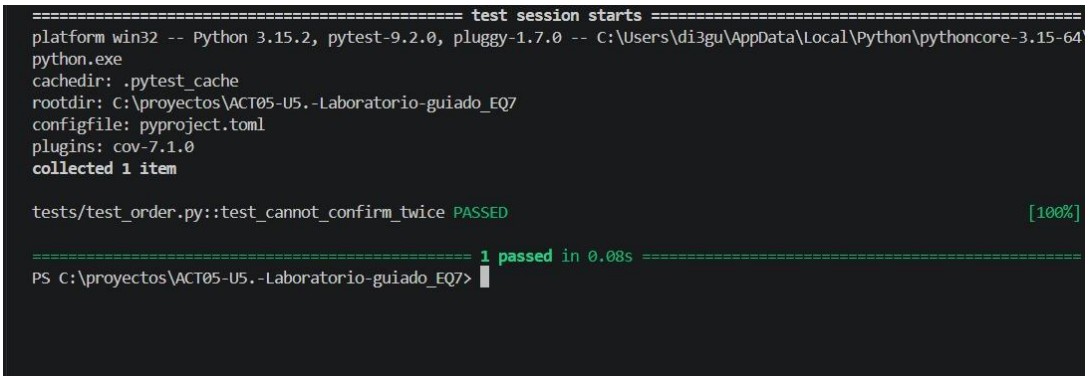


Figura 9. E3 – fase GREEN: 1 passed

3. Fase REFACTOR (mejora de diseño y encapsulamiento)

Se limpian y extraen las validaciones hacia el método auxiliar `_ensure_confirmable(customer_debt)` para modularizar el dominio sin alterar el comportamiento observable:

```
def confirm(self, customer_debt=0):
    self._ensure_confirmable(customer_debt)
    self.discount = round(self.total * VIP_DISCOUNT_RATE, 2) if self.vip else 0.0
    self.status = "CONFIRMED"
    return self.total - self.discount
```

Dato	Valor
Verificación de regresión global	Se ejecuta la suite completa del proyecto para garantizar que el refactor no rompa contratos existentes
Métrica global post-refactor	47 passed, 1 xfailed (coincidente con las evidencias de E1)

4. Fundamento metodológico: TDD vs. Test-Last

Aspecto	Qué aporta TDD
Validación de la prueba	La fase RED confirma que el defecto existe en el sistema y que la prueba es sensible a él (`DID NOT RAISE ValueError`). Escribir la prueba después del código no ofrece certeza de que realmente falle ante una regresión
Diseño guiado	En GREEN, el desarrollo se limita al código estrictamente necesario para pasar el caso
Red de seguridad	En REFACTOR, la suite de pruebas completa protege las reestructuraciones internas sin riesgo de alterar las reglas de negocio

E4 · Persistencia, commit y rollback

```
def test_repository_persists_second_order():
    engine = make_engine()
    with Session(engine) as session:
        SqlOrderRepository(session).save(2, Order(250))
        session.commit()
        row = session.scalar(select(OrderRow).where(OrderRow.id == 2))
        assert (row.id, row.total, row.status) == (2, 250, "NEW")
```

Comando: `python -m pytest tests/test_repository.py -v`

Operación	Efecto en la base	Prueba / resultado
`commit`	Hace permanentes los cambios de la transacción	Fila id=2, total 250, estado NEW · PASSED
`rollback`	Descarta los cambios; la base queda como antes	Consulta de id=1 devuelve None · PASSED
Restricción de unicidad	Un segundo pedido con `id=1` lanza `IntegrityError`	Se conserva el original (total 120) · PASSED

Todo ocurre en SQLite en memoria, sin datos reales.

```
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7> python -m pytest tests/test_repository.py -v
========================================= test session starts =========================================
platform win32 -- Python 3.15.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\
python.exe
cachedir: .pytest_cache
rootdir: C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7
configfile: pyproject.toml
plugins: cov-7.1.0
collected 4 items

tests/test_repository.py::test_repository_commit PASSED                                  [ 25%]
tests/test_repository.py::test_repository_rollback PASSED                                [ 50%]
tests/test_repository.py::test_repository_persists_second_order PASSED                   [ 75%]
tests/test_repository.py::test_repository_rejects_duplicate_id_and_keeps_original PASSED [100%]

========================================== 4 passed in 0.71s ==========================================
```

Figura 10. E4 – pruebas del repositorio, 4 passed

E5 · Pasarela de pago con mocks

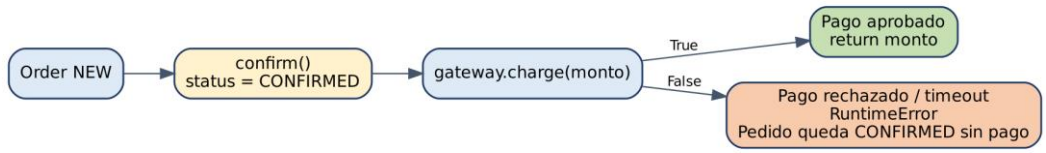


Figura 11. Flujo de checkout y riesgo de consistencia detectado

Escenario	Configuración del mock	Verificación	Resultado
Pago aprobado	La pasarela devuelve True (total 100)	Devuelve el importe; `assert_called_once_with(100)`	PASSED
Pago rechazado, VIP de 200	La pasarela devuelve False	`RuntimeError("payment rejected")`; intento de cobro por 170	PASSED
Timeout	La pasarela lanza `TimeoutError`	El error se propaga al llamador	PASSED
Error de dominio (total 0)	No se configura respuesta	`charge` no se llama (`assert_not_called`)	PASSED
Pedido con pago rechazado no debe quedar confirmado	La pasarela devuelve False	Estado final distinto de CONFIRMED	XFAIL

El mock usa `Mock(spec=["charge"])`: solo existe `charge`, no hay red ni dinero real. Comando: `python -m pytest tests/test_service.py -v -rx`

```
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7>  python -m pytest tests/test_service.py -v -rx
============================================= test session starts =============================================
platform win32 -- Python 3.15.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\
python.exe
cachedir: .pytest_cache
rootdir: C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7
configfile: pyproject.toml
plugins: cov-7.1.0
collected 6 items

tests/test_service.py::test_gateway_approved PASSED                                                  [ 16%]
tests/test_service.py::test_gateway_rejected PASSED                                                  [ 33%]
tests/test_service.py::test_gateway_rejected_for_vip_200_charges_170 PASSED                          [ 50%]
tests/test_service.py::test_gateway_timeout_propagates PASSED                                        [ 66%]
tests/test_service.py::test_domain_error_never_reaches_gateway PASSED                                [ 83%]
tests/test_service.py::test_rejected_payment_should_not_leave_order_confirmed XFAIL (RIESGO CONOCIDO: chec...) [100%]

============================================= short test summary info =============================================
XFAIL tests/test_service.py::test_rejected_payment_should_not_leave_order_confirmed - RIESGO CONOCIDO: checkout confir
ma el pedido antes de saber si el pago fue aceptado
========================================== 5 passed, 1 xfailed in 0.22s ===========================================
```

Figura 12. E5 – pruebas del servicio con mock, 5 passed y 1 xfailed

Riesgo de consistencia (sin ocultarlo). `checkout` ejecuta `order.confirm()` antes de cobrar. Si la pasarela rechaza o expira, el objeto queda en CONFIRMED sin pago. La prueba `test_rejected_payment_should_not_leave_order_confirmed` lo documenta con `xfail(strict=True)`.

Opción	Idea de corrección
A	Calcular el importe sin cambiar el estado, cobrar y marcar CONFIRMED solo si el cobro fue exitoso
B	Estado intermedio `PENDING_PAYMENT`; pasa a CONFIRMED al aprobarse el pago
C	Compensación: si el cobro falla, restaurar el estado anterior y no hacer commit

E6 · Seguridad y autorización

Reglas implementadas: `SHIPPED` nunca se cancela (ni por administrador); para otros estados el administrador puede cancelar; el dueño no administrador solo durante las primeras 24 h (estrictamente menos de 24 h). Las pruebas usan una fecha fija y pasan `now` explícito.

Rol	¿Es el dueño?	Estado	Antigüedad	Resultado	Prueba
Dueño	Sí	NEW	1 h	Permitido	`dueno-1h-NEW`
Dueño	Sí	NEW	23 h 59 min	Permitido	`dueno-23h59-NEW`
Dueño	Sí	NEW	24 h	Denegado	`dueno-24h-NEW`
Dueño	Sí	NEW	25 h	Denegado	`dueno-25h-NEW`
Tercero	No	NEW	1 h	Denegado	`tercero-1h-NEW`
Administrador	No	NEW	100 h	Permitido	`admin-tercero-NEW`
Dueño	Sí	CONFIRMED	1 h	Permitido	`dueno-1h-CONFIRMED`
Dueño	Sí	SHIPPED	1 h	Denegado	`dueno-SHIPPED`
Administrador	No	SHIPPED	1 h	Denegado	`admin-SHIPPED`

Comando: `python -m pytest tests/test_security.py -v`

```
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7> python -m pytest tests/test_security.py -v
========================================= test session starts =========================================
platform win32 -- Python 3.15.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\
python.exe
cachedir: .pytest_cache
rootdir: C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7
configfile: pyproject.toml
plugins: cov-7.1.0
collected 16 items

tests/test_security.py::test_owner_within_24h PASSED                                    [  6%]
tests/test_security.py::test_stranger_denied PASSED                                     [ 12%]
tests/test_security.py::test_shipped_denied_even_admin PASSED                           [ 18%]
tests/test_security.py::test_expired_denied PASSED                                      [ 25%]
tests/test_security.py::test_admin_cancels_new_order_of_another_user PASSED             [ 31%]
tests/test_security.py::test_owner_at_exactly_24h_denied PASSED                         [ 37%]
tests/test_security.py::test_default_now_is_used_when_not_provided PASSED               [ 43%]
tests/test_security.py::test_authorization_matrix[dueno-1h-NEW] PASSED                  [ 50%]
tests/test_security.py::test_authorization_matrix[dueno-23h59-NEW] PASSED               [ 56%]
tests/test_security.py::test_authorization_matrix[dueno-24h-NEW] PASSED                 [ 62%]
tests/test_security.py::test_authorization_matrix[dueno-25h-NEW] PASSED                 [ 68%]
tests/test_security.py::test_authorization_matrix[tercero-1h-NEW] PASSED                [ 75%]
tests/test_security.py::test_authorization_matrix[admin-tercero-NEW] PASSED             [ 81%]
tests/test_security.py::test_authorization_matrix[dueno-1h-CONFIRMED] PASSED            [ 87%]
tests/test_security.py::test_authorization_matrix[dueno-SHIPPED] PASSED                 [ 93%]
tests/test_security.py::test_authorization_matrix[admin-SHIPPED] PASSED                 [100%]

========================================== 16 passed in 0.13s ==========================================
```

Figura 13. E6 – pruebas de autorización, 16 passed

E7 · Cobertura y calidad

Comando	Resultado
`python -m ruff check src tests`	All checks passed
`python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80`	47 passed, 1 xfailed; cobertura 100 % (55 sentencias); umbral de 80 % alcanzado

```
All checks passed!
.............................................x                                          [100%]
---------- coverage: platform win32, python 3.15.2-final-0 ----------

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
src\__init__.py         0      0   100%
src\application.py      8      0   100%
src\domain.py          21      0   100%
src\repository.py      18      0   100%
src\security.py         8      0   100%
-------------------------------------------------
TOTAL                  55      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
47 passed, 1 xfailed in 1.21s
C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\Lib\site-packages\_pytest\unraisableexception.py:29: ResourceWa
rning: unclosed database in <sqlite3.Connection object at 0x000002162DAFADC0>
  gc.collect()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
C:\Users\di3gu\AppData\Local\Python\pythoncore-3.15-64\Lib\site-packages\_pytest\unraisableexception.py:29: ResourceWa
rning: unclosed database in <sqlite3.Connection object at 0x000002162DC202E0>
  gc.collect()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
PS C:\proyectos\ACT05-U5.-Laboratorio-guiado_EQ7>
```

Figura 14. E7 – ruff sin errores y resumen de cobertura en terminal: 100 %



Figura 15. E7 – reporte HTML de cobertura (htmlcov/index.html): 100 %, 55 sentencias

La cobertura de líneas es 100 %, pero eso no significa que no haya comportamientos sin probar. Ejemplos reales que ninguna prueba ejercita:

Comportamiento sin probar	Consecuencia
`can_cancel` con `created_at` sin zona horaria	Lanza `TypeError` (no se puede restar fecha ingenua y fecha con zona); no hay regla ni prueba que defina qué debe pasar
`OrderRow` sin columna de descuento	El repositorio guarda el total sin descuento y el estado, no el importe cobrado
`customer_debt` negativo	Se trata como "sin deuda"; no hay prueba que fije ese comportamiento

Observación. Al terminar la ejecución aparecen dos `ResourceWarning` por conexiones SQLite que no se cierran explícitamente en las pruebas de integración. No afectan los resultados; la mejora sería cerrar el motor (`engine.dispose()`) al final de cada prueba.

E8 · GitHub Actions

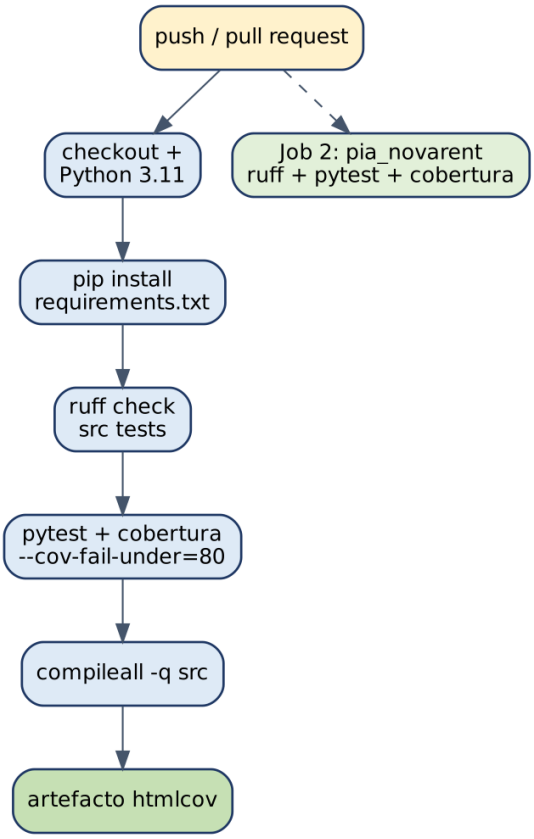


Figura 16. Pasos del workflow CI Business Logic

Elemento	Detalle
Workflow	`CI Business Logic` (`.github/workflows/ci.yml`)
Disparadores	Cada `push` y `pull_request`
Job `test`	checkout → Python 3.11 → instalar dependencias → `ruff` → `pytest` con cobertura y umbral 80 % → `compileall` → artefacto `htmlcov`
Job `pia_novarent`	`ruff` y `pytest` con cobertura sobre la carpeta `pia_novarent/` (E9)
Repositorio	https://github.com/cesarmax90/ACT05-U5.-Laboratorio-guiado_EQ7

Simulación local del ciclo verde → rojo → verde con los mismos pasos del workflow (`scripts/ci_local.sh`):

Commit	Estado	Resultado de la ejecución	Código de salida
`1db7138` Unidad 5: pruebas y CI	Verde	All checks passed; cobertura 100 %; 47 passed, 1 xfailed	0
`3e9fcdc` ROMPER: aserción falsa (fallo controlado)	Rojo	`test_vip_discount` FAILED: `assert 85.0 == 86`	1
`1883e1f` Revert del commit defectuoso	Verde	All checks passed; cobertura 100 %; 47 passed, 1 xfailed	0

Qué detecta el pipeline. Con una aserción falsa (`assert o.confirm() == 85` cambiada a `== 86`), `pytest` termina con código de salida 1, el job falla y el cambio defectuoso queda marcado en rojo; al revertir el commit el pipeline regresa a verde. La protección obligatoria de merge requeriría configurar branch protection o rulesets en GitHub; el YAML por sí solo no la activa.

Ejecuciones en GitHub Actions

#	Ejecución	Descripción
1	Verde	Proyecto original, todas las verificaciones pasan
2	Roja	Aserción falsa introducida a propósito; el pipeline detecta el fallo
3	Verde	Tras revertir el cambio defectuoso

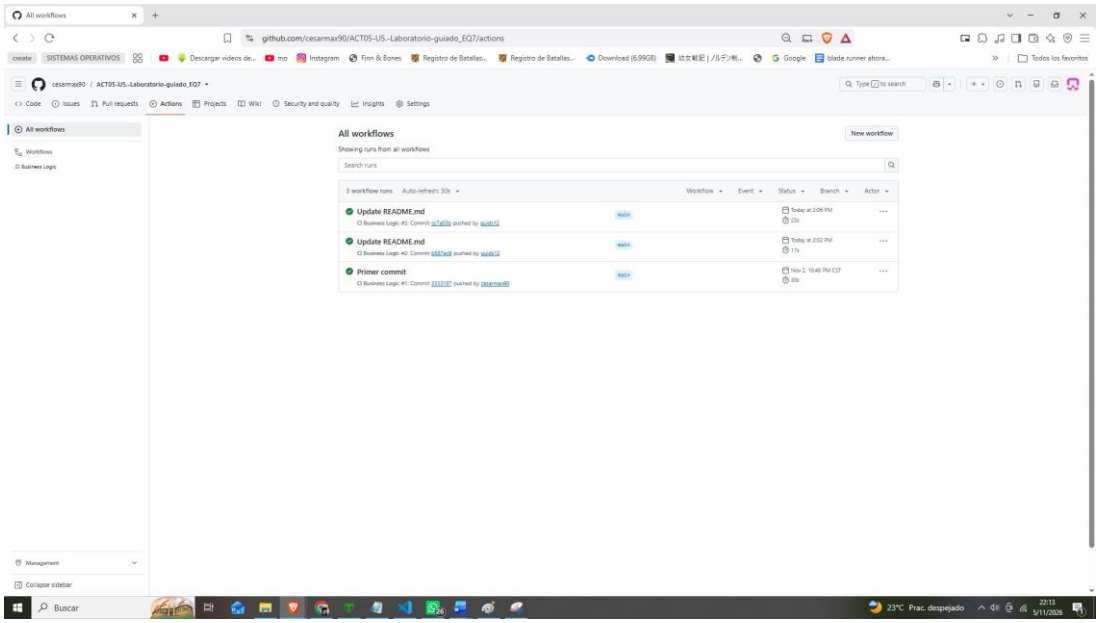


Figura 17. E8 – ejecución verde en GitHub Actions

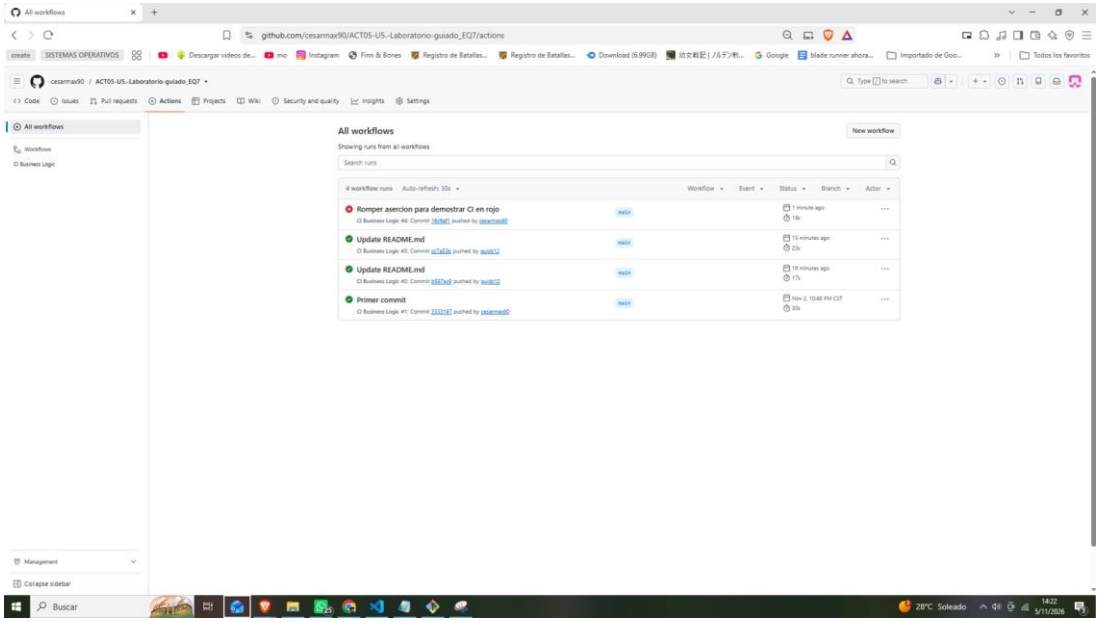


Figura 18. E8 – ejecución roja: el pipeline detecta la aserción falsa

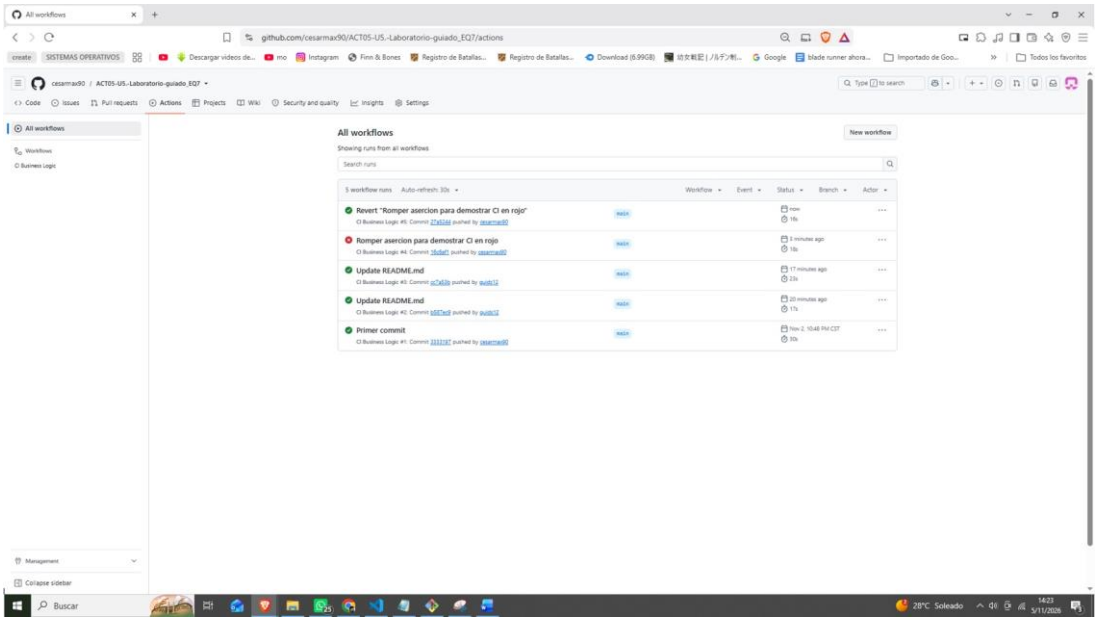


Figura 19. E8 – ejecución verde tras revertir el cambio defectuoso

7. Matriz de casos de prueba del laboratorio

Archivo: `datos/matriz_casos_prueba.csv` (20 casos: 7 éxito, 9 error y 4 límite; todos PASSED).

ID	Tipo	Regla	Entrada	Resultado esperado	Resultado
CP-01	Éxito	VIP recibe 15 % de descuento	Total=100, VIP	CONFIRMED, cobro 85	PASSED
CP-02	Éxito	VIP recibe 15 % de descuento	Total=200, VIP	CONFIRMED, cobro 170, descuento 30	PASSED
CP-03	Éxito	Cliente regular sin descuento	Total=200, regular	CONFIRMED, cobro 200	PASSED
CP-04	Éxito	Commit persiste el pedido	save(id=2, total=250) + commit	Fila id=2, total 250, estado NEW	PASSED
CP-05	Éxito	Rollback descarta el pedido	save(id=1) + rollback	Consulta id=1 devuelve None	PASSED
CP-06	Éxito	Pago aprobado (mock)	Pasarela devuelve True, total=100	Cobro 100, una sola llamada	PASSED
CP-07	Éxito	Admin cancela pedido NEW ajeno	NEW, admin, usuario distinto del dueño	Permitido	PASSED
CP-08	Error	Cliente con deuda no confirma	Total=100, deuda=50	CustomerHasDebt	PASSED
CP-09	Error	Total debe ser positivo	Total=0 y Total=-1	ValueError	PASSED
CP-10	Error	No confirmar dos veces (TDD)	Confirmar un pedido ya CONFIRMED	ValueError already confirmed	PASSED
CP-11	Error	Pago rechazado (mock)	VIP 200, pasarela devuelve False	RuntimeError; intento de cobro por 170	PASSED
CP-12	Error	Timeout de la pasarela	Pasarela lanza TimeoutError	TimeoutError se propaga	PASSED

ID	Tipo	Regla	Entrada	Resultado esperado	Resultado
CP-13	Error	Error de dominio no cobra	Total=0	ValueError y charge() sin llamar	PASSED
CP-14	Error	Unicidad de id	Dos pedidos con id=1	IntegrityError; se conserva el original (120)	PASSED
CP-15	Error	Tercero no cancela	NEW, usuario distinto del dueño, no admin	Denegado	PASSED
CP-16	Error	SHIPPED nunca se cancela	SHIPPED, admin	Denegado	PASSED
CP-17	Límite	Dueño a exactamente 24 h	NEW, dueño, antigüedad 24:00	Denegado	PASSED
CP-18	Límite	Dueño justo antes de 24 h	NEW, dueño, antigüedad 23:59	Permitido	PASSED
CP-19	Límite	Importe mínimo con descuento redondeado	Total=0.01, VIP	CONFIRMED, cobro 0.01	PASSED
CP-20	Límite	Redondeo del descuento	Total=99.99, VIP	CONFIRMED, cobro 84.99	PASSED

8. E9 · Aplicación al PIA (NovaRent)

La empresa del PIA es **NovaRent** (renta de autos por día). Las reglas se tomaron del BRD, las User Stories y el diseño arquitectural de las Unidades 2 y 3. El código está en la carpeta `pia_novarent/`, autocontenida (con su propio `README.md`, `pyproject.toml`, pruebas y matriz), de modo que se pueda trasladar al repositorio del PIA cuando este se cree. El pipeline la ejecuta en el job `pia_novarent`.

8.1 Reglas seleccionadas

Regla	US	Parámetros (BRD / Unidad 3)	Clase
Cargo por kilometraje excedente	US14	200 km/día incluidos, $3 MXN por km extra	`PoliticaCargos`
Autorización de extensión por rol	US11, US12	Agente hasta 48 h; más de 48 h requiere Administración; solicitud con 2 h o más de anticipación	`PoliticaExtension`
Cancelación y no-show	US03, US04	Cancelación gratuita con 24 h o más; no-show se libera a las 2 h	`PoliticaCancelacion`

8.2 Matriz de casos de prueba

Archivo: `pia_novarent/datos/matriz_casos_pia.csv` (20 casos: 6 éxito, 6 error y 8 límite). Cada caso enlaza con su prueba por el id `CP-PIA-NN`.

ID	Regla	Entrada	Resultado esperado	Tipo	Resultado
CP-PIA-01	Km excedente (US14): 200 km/día incluidos, $3 por km extra	3 días, 750 km	Cargo $450 (150 km excedentes)	Éxito	PASSED
CP-PIA-02	Km excedente (US14): el kilometraje no puede ser negativo	1 día, -1 km	ValueError	Error	PASSED

ID	Regla	Entrada	Resultado esperado	Tipo	Resultado
CP-PIA-03	Km excedente (US14): exactamente lo incluido no genera cargo	1 día, 200 km	Cargo $0	Límite	PASSED
CP-PIA-04	Km excedente (US14): 1 km sobre lo incluido	1 día, 201 km	Cargo $3	Límite	PASSED
CP-PIA-05	Extensión (US11): el agente aprueba hasta 48 h	Agente, +24 h, vence en 10 h	APROBADA; vencimiento +24 h	Éxito	PASSED
CP-PIA-06	Extensión (US11): se pide con 2 h de anticipación mínima	Agente, +24 h, vence en 1 h 58 min	ExtensionFueraDeTiempo	Error	PASSED
CP-PIA-07	Extensión (US11): 48 h exactas las aprueba el agente	Agente, +48 h, vence en 10 h	APROBADA	Límite	PASSED
CP-PIA-08	Extensión (US12): más de 48 h requiere Administración	Agente, +49 h, vence en 10 h	REQUIERE_AUTORIZACION; vencimiento sin cambio	Límite	PASSED
CP-PIA-09	Extensión (US11): anticipación de exactamente 2 h es válida	Agente, +24 h, vence en 2 h	APROBADA	Límite	PASSED
CP-PIA-10	Cancelación (US03): gratis con 24 h o más de anticipación	Cancelar 48 h antes del inicio	Estado Cancelada, sin penalización	Éxito	PASSED
CP-PIA-11	Cancelación (US03): no se cancela dos veces	Reservación ya Cancelada	ValueError	Error	PASSED
CP-PIA-12	Cancelación (US03): 24 h exactas no penalizan	Cancelar 24 h antes del inicio	Estado Cancelada, sin penalización	Límite	PASSED
CP-PIA-13	Cancelación (US03): 23 h 59 min sí penalizan	Cancelar 23 h 59 min antes	Estado Cancelada, con penalización	Límite	PASSED
CP-PIA-14	No-show (US04): se libera pasadas 2 h	3 h después del inicio	Estado Liberada	Éxito	PASSED
CP-PIA-15	No-show (US04): no se libera dentro de la ventana	1 h 58 min después del inicio	ValueError; sigue Creada	Error	PASSED
CP-PIA-16	No-show (US04): exactamente 2 h se libera	2 h después del inicio	Estado Liberada	Límite	PASSED
CP-PIA-17	Persistencia: la reservación sobrevive al commit	Guardar R-100 y hacer commit	Se recupera con los mismos datos	Éxito	PASSED
CP-PIA-18	Persistencia: rollback descarta la reservación	Guardar R-100 y hacer rollback	obtener_por_folio devuelve None	Error	PASSED
CP-PIA-19	Pasarela (mock): depósito aprobado abre el contrato	retener_deposito devuelve DEP-OK-1	Contrato Abierto; pasarela llamada 1 vez con 5000	Éxito	PASSED
CP-PIA-20	Pasarela (mock): depósito rechazado no abre el contrato	retener_deposito devuelve None	RuntimeError; contrato sigue Pendiente	Error	PASSED

Los casos CP-PIA-17 y CP-PIA-18 cubren la persistencia (SQLite en memoria, commit y rollback); CP-PIA-19 y CP-PIA-20 cubren el servicio externo: la `PasarelaPago` se sustituye por un

`Mock(spec=["retener_deposito"])`, por lo que no hay cobros reales. Además de la matriz hay pruebas de unicidad del folio, timeout de la pasarela y rol sin permiso.

8.3 Ejecución y cobertura

Comando (desde `pia_novarent/`)	**Resultado**
`python -m ruff check novarent tests`	Sin errores
`python -m pytest -v --cov=novarent --cov-report=term-missing --cov-report=html --cov-fail-under=80`	31 pruebas PASSED; cobertura 100 % sobre `novarent/` (123 sentencias; umbral 80 %)

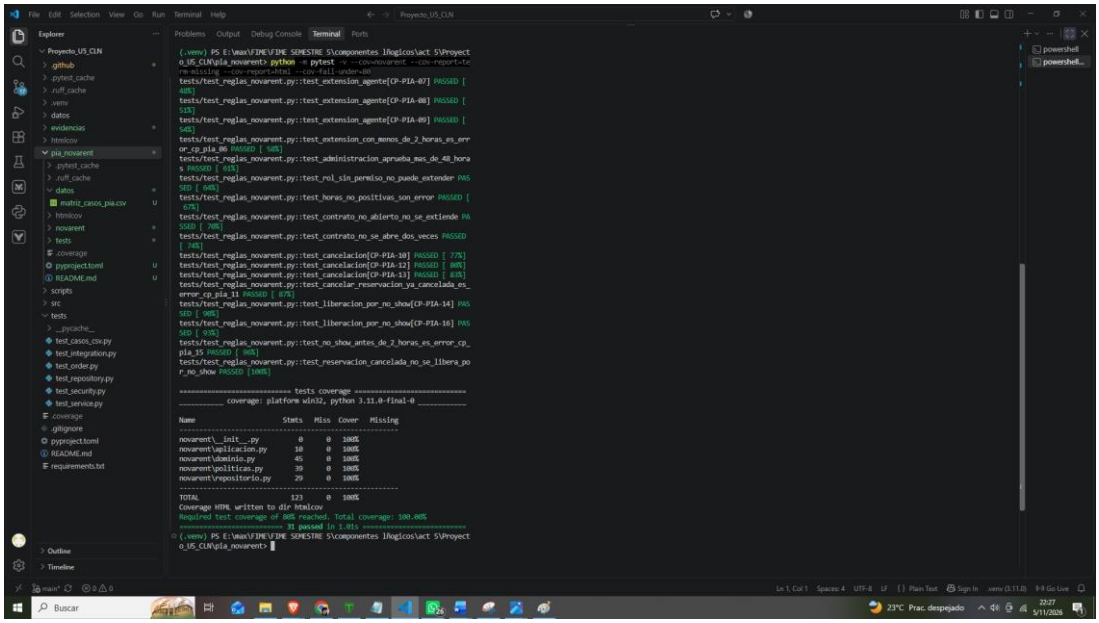


Figura 20. E9 – ruff, pruebas y cobertura de pia_novarent en terminal

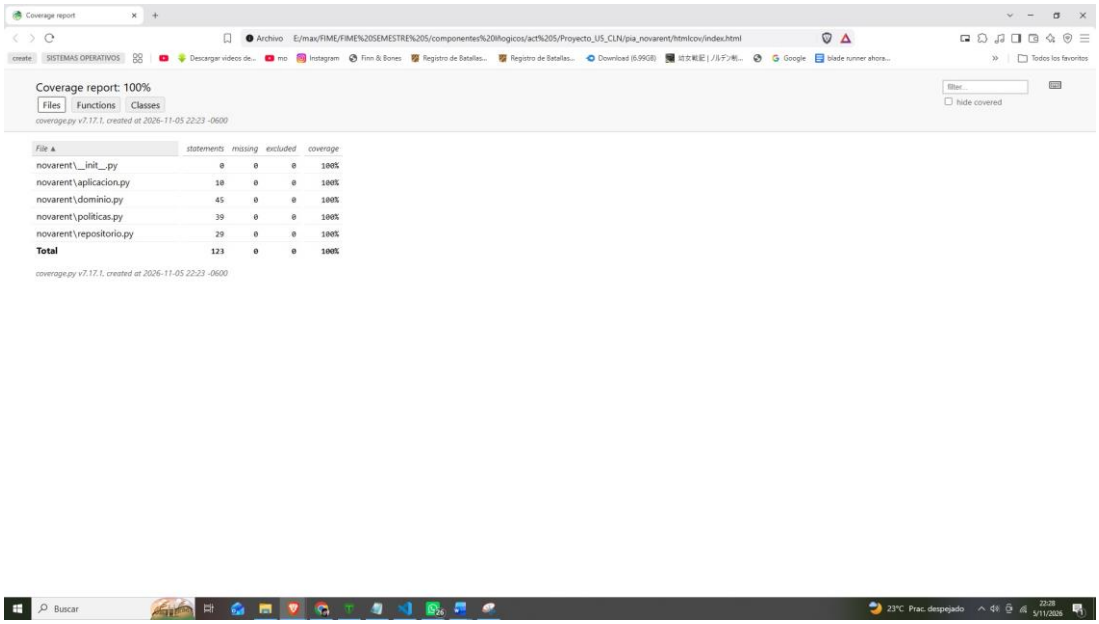


Figura 21. E9 – reporte HTML de cobertura de pia_novarent

Defecto inyectado a propósito	Casos que fallaron	Resultado
Tarifa por km de $3 a $4	CP-PIA-01 y CP-PIA-04	3 failed, 28 passed (en conjunto con el defecto siguiente)
Límite del agente de 48 h a 72 h	CP-PIA-08	`E9_defecto_detectado.txt`

8.4 Decisiones de diseño

Decisión	Justificación
Políticas parametrizables y puras	Los valores de negocio (200 km, $3, 48 h, 2 h, 24 h) se reciben por constructor; cambiar una política no toca los servicios ni el dominio
El dominio no conoce SQLAlchemy ni la pasarela	Las reglas se prueban sin base de datos; la persistencia vive en `repositorio.py` y la pasarela es un puerto sustituible
Depósito primero, contrato después	`abrir_contrato` retiene el depósito antes de abrir el contrato; un rechazo o timeout deja el contrato en `Pendiente`. Evita el riesgo de consistencia documentado en E5
Fechas fijas	Todas las pruebas pasan `ahora` explícito; el resultado nunca depende del reloj
Roles	Un rol distinto de Agente o Administración recibe `PermissionError`
Fuera de alcance por datos incompletos	El monto de la penalización por cancelación tardía y las tarifas de retraso y de combustible por octavo no están definidos en el BRD, por lo que no se inventaron. Las cifras deben validarse con el cliente antes de producción

9. Registro de uso de IA

Campo	Detalle
Prompt usado	Con los documentos del laboratorio y de las Unidades 2 y 3 como contexto: (1) revisar el repositorio frente a las instrucciones del laboratorio; (2) completar `EVIDENCIAS_U5.md` con datos del equipo, capturas y reflexión final; (3) a partir del BRD, las User Stories y el diseño arquitectural de NovaRent, proponer tres reglas y escribir pruebas pytest de éxito, error y límite, con persistencia en SQLite y mock de la pasarela de pago
Caso sugerido y corregido	Tomar el resultado esperado de CP08 desde el CSV (`CONFIRMED`) habría hecho fallar la prueba después del TDD
Corrección manual	`ESPERADO_TRAS_TDD = {"CP08": "ValueError"}` en `tests/test_casos_csv.py`
Prueba añadida	`test_cannot_confirm_twice` y la parametrización desde `datos/casos_de_prueba.csv`

10. Reflexión final

Defecto más costoso	Por qué importa	Prueba que lo evita
Dejar confirmado un pedido cuyo pago fue rechazado	`checkout` confirma antes de saber si la pasarela aceptó el cobro; si el pago se rechaza o expira, la empresa entregaría mercancía sin cobrarla	`test_rejected_payment_should_not_leave_order_confirmed` (mock de la pasarela; `xfail` estricta para dejar visible el riesgo hasta que se corrija)
Confirmar dos veces el mismo pedido	Podría duplicar cobros	`test_cannot_confirm_twice` (surgió del ciclo TDD)

También comprobamos que el 100 % de cobertura no demuestra corrección: el riesgo del checkout existía con todas las líneas ejecutadas. Lo que da valor a una prueba son sus aserciones y los escenarios que cubre, no el porcentaje. El pipeline de integración continua evita que un cambio posterior rompa estas reglas sin que el equipo se entere.

11. Autoevaluación

Criterio	Cumple	Dónde se demuestra
Una prueba unitaria no requiere base de datos (dominio y seguridad se prueban sin SQLAlchemy)	✓	E2, E6
Evidencia real de RED y GREEN	✓	E3
Diferencia entre commit y rollback	✓	E4
El mock no genera cargo real (`Mock(spec=["charge"])`)	✓	E5
Permisos y límites temporales comprobados con fechas fijas	✓	E6
Cobertura ≥ 80 % sin ocultar módulos (`src/` completo)	✓	E7
Pipeline ejecutado en GitHub y fallo detectado	✓	E8
Casos del PIA con reglas reales de la empresa: 20 casos de NovaRent, más de los nueve requeridos	✓	E9

12. Repositorio

https://github.com/cesarmax90/ACT05-U5.-Laboratorio-guiado_EQ7